

Universidad de San Carlos de Guatemala
Centro Universitario de Occidente
División de Ciencias de la Ingeniería
Estructura de Datos



Catálogo de Productos de Supermercado

Proyecto Fase 1 – Estructura de Datos

Nombre: Juan Fernando García Natareno
RA: 202230077
Primer Semestre 2026

El presente proyecto corresponde a la evolucion del sistema de gestion de catalogo de productos desarrollado en la Fase 1, ampliandose hacia un modelo distribuido basado en multiples sucursales interconectadas. El sistema esta desarrollado completamente en JavaScript con Programacion Orientada a Objetos, sin librerias que implementen las estructuras de datos.

Cada sucursal administra su propio inventario de productos utilizando las mismas estructuras de datos avanzadas de la Fase 1: listas enlazadas, arbol AVL, arbol B, arbol B+ y tabla hash. La novedad principal de esta fase es la introduccion de un grafo ponderado que representa la red de sucursales, donde cada nodo corresponde a una sucursal y cada arista representa una conexion con peso de tiempo y costo de traslado.

El sistema implementa el algoritmo de Dijkstra para calcular rutas optimas de transferencia de productos entre sucursales, simulando el flujo real de despacho mediante estructuras de colas. Ademias se mantiene el mecanismo de insercion atomizada con rollback mediante una pila (Stack) implementada desde cero.

Diseno del Sistema

Arquitectura General

El sistema sigue una arquitectura en capas separadas por responsabilidad:

Capa	Descripcion
Estructuras de datos	Clases puras sin dependencias del DOM. Implementan AVL, B, B+, Hash, Cola, Pila y Lista enlazada.
Logica de negocio	Catalogo orquesta las 6 estructuras. Rama (Branch) encapsula cada sucursal con su catalogo y sus 3 colas.
Persistencia	Storage.js gestiona el sessionStorage del navegador para compartir datos entre paginas.
Presentacion	Un archivo JS por pagina HTML. Sin frameworks. Bootstrap para estilos.

Clase Producto

Cada producto contiene 9 campos: nombre (string), codigoBarras (string, clave unica), categoria (string), fechaExpiracion (string YYYY-MM-DD), marca (string), precio (double), stock (int), estado (string: Disponible/En transito/Agotado) y sucursalId (string). Los operadores de comparacion estan implementados como metodos: lessThan() y greaterThan() comparan por nombre (para el AVL y lista ordenada), equals() compara por nombre exacto.

Estructuras de datos implementadas

a. Lista enlazada no ordenada

Implementacion clasica de lista enlazada simple con nodo cabeza. La insercion se realiza al frente en $O(1)$. La busqueda recorre todos los nodos secuencialmente en $O(n)$. Soporta busqueda por nombre y por codigo de barras. Se incluye para el benchmark comparativo que demuestra empiricamente la ventaja del AVL y la tabla hash.

b. Lista enlazada ordenada

Variante de la lista enlazada que mantiene los productos ordenados alfabeticamente por nombre. La insercion recorre la lista para encontrar la posicion correcta en $O(n)$. La ventaja es el corte temprano en busquedas: si el nodo actual ya es mayor al buscado, se detiene sin recorrer el resto.

c. Arbol AVL

Arbol binario de busqueda auto-balanceado con clave por nombre del producto. Garantiza que el factor de balance (altura izquierda menos altura derecha) de cada nodo sea -1, 0 o +1. Se implementan 4 casos de rotacion: LL (rotacion simple derecha), RR (rotacion simple izquierda),

LR (rotacion doble) y RL (rotacion doble). El recorrido in-order produce los productos ordenados alfabeticamente.

d. Tabla Hash

Tabla hash con encadenamiento (chaining) para manejo de colisiones. La clave es el codigo de barras. Utiliza una funcion hash polinomial con base 31 y capacidad primo (1009) para una distribucion uniforme. Cada bucket contiene una lista enlazada de NodoHash. La insercion se realiza al inicio de la cadena. El factor de carga se monitorea para evaluar la calidad de la distribucion.

e. Arbol B

Arbol B de grado minimo $t=3$, donde cada nodo puede tener entre 2 y 5 claves y entre 3 y 6 hijos. La clave de ordenamiento es la fecha de caducidad en formato ISO (YYYY-MM-DD), lo que permite comparacion lexicografica directa. Cuando un nodo se llena ($2t-1=5$ claves), se divide: la clave del medio sube al padre. La busqueda por rango es la operacion principal: dado un intervalo [fechaInicio, fechaFin], recorre el arbol encontrando todos los productos cuya fecha este en ese rango.

f. Arbol B+

Arbol B+ de grado minimo $t=3$ con clave por categoria del producto. A diferencia del arbol B, los nodos internos solo contienen claves de navegacion mientras que todos los datos residen en las hojas. Las hojas estan enlazadas entre si mediante un puntero siguiente, formando una lista enlazada. Permite claves duplicadas (multiples productos de la misma categoria). La busqueda navega hasta la hoja correcta en $O(\log n)$ y recorre el enlace horizontal recolectando todos los productos de esa categoria en $O(k)$.

g. Cola (Queue)

Cola FIFO implementada con lista enlazada. Cada sucursal tiene 3 colas: cola de ingreso (productos recién recibidos), cola de traspaso (productos en preparacion para envio) y cola de salida (productos listos para despachar). El sistema respeta el orden FIFO y el intervalo de despacho configurado por sucursal.

h. Pila (Stack)

Pila LIFO implementada con lista enlazada. Se usa para gestionar el historial de operaciones del Catalogo. Cada operacion de agregar o eliminar producto se guarda en la pila, permitiendo deshacer (rollback) la ultima operacion en cualquier momento, revirtiendo los cambios en las 6 estructuras simultaneamente.

i. Grafo ponderado + Dijkstra

Grafo dirigido con lista de adyacencia. Cada nodo es una sucursal (Rama) y cada arista es una conexion con peso de tiempo (segundos) y costo (quetzales). Soporta conexiones bidireccionales y unidireccionales. Implementa el algoritmo de Dijkstra con matriz de adyacencia para calcular la ruta optima entre dos sucursales segun el criterio elegido por el usuario: tiempo minimo o menor costo.

Analisis de Complejidad (Big-O)

Estructura	Insercion	Busqueda	Eliminacion	Uso principal
Lista no ordenada	$O(1)$	$O(n)$	$O(n)$	Benchmark comparativo
Lista ordenada	$O(n)$	$O(n)^*$	$O(n)$	Listado alfabetico
Arbol AVL	$O(\log n)$	$O(\log n)$	$O(\log n)$	Busqueda por nombre
Tabla Hash	$O(1)$ prom.	$O(1)$ prom.	$O(1)$ prom.	Busqueda por codigo
Arbol B	$O(\log n)$	$O(\log n + k)$	$O(\log n)$	Busqueda por fecha
Arbol B+	$O(\log n)$	$O(\log n + k)$	$O(\log n)$	Busqueda por categoria
Cola (Queue)	$O(1)$	—	$O(1)$	Colas de despacho
Pila (Stack)	$O(1)$	—	$O(1)$	Rollback de operaciones
Grafo + Dijkstra	$O(1)$	$O(V^2)$	$O(E)$	Red de sucursales

Justificacion de las complejidades

AVL $O(\log n)$: Al mantener el factor de balance entre -1 y $+1$, la altura del arbol siempre es $O(\log n)$. Cada operacion recorre un camino desde la raiz hasta una hoja, mas las rotaciones que son $O(1)$ cada una.

Hash $O(1)$ promedio: La funcion hash distribuye las claves uniformemente. Con capacidad 1009 (primo) y factor de carga aproximado de 1.0, la busqueda toma tiempo constante en promedio. El peor caso $O(n)$ ocurre solo si todas las claves colisionan en el mismo bucket.

Arbol B $O(\log n)$: Con grado $t=3$, cada nodo interno tiene entre 3 y 6 hijos. La altura es $O(\log_t n)$. La busqueda por rango agrega $O(k)$ por los k resultados encontrados.

Arbol B+ $O(\log n + k)$: Similar al B, pero las hojas enlazadas permiten recorrer resultados consecutivos sin subir y bajar por el arbol, haciendo el recorrido por categoria mas eficiente.

Dijkstra $O(V^2)$: Implementacion con matriz de adyacencia y busqueda lineal del minimo. Para el numero de sucursales esperado (menos de 100), esta complejidad es completamente aceptable.

Logica del Sistema

La logica del negocio esta organizada en la carpeta src/Logica/ con los siguientes archivos:

- **Catalogo.js:** Nucleo del sistema. Contiene internamente las 6 estructuras de datos. Implementa insercion atomizada con rollback: si una insercion falla en la estructura N, se deshacen las inserciones exitosas en las estructuras 1 a N-1. Usa una pila (Stack) para gestionar el historial de operaciones.
- **Storage.js:** Capa de persistencia. Gestiona el sessionStorage del navegador para compartir datos entre paginas. Provee funciones para guardar y recuperar sucursales, conexiones, productos e historial de transferencias.
- **CSV.js:** Encargado de la lectura de archivos CSV usando la FileReader API del navegador. Implementa validaciones para cada tipo de archivo (sucursales, conexiones, productos). Genera archivos de log de errores descargables: errores_sucursales.txt, errores_conexiones.txt y errores_productos.txt.
- **Stack.js:** Implementacion de la pila LIFO con lista enlazada. Usada por el Catalogo para el historial de operaciones de rollback.

Sistema de Despacho y Red de Sucursales

Cuando un producto debe trasladarse de una sucursal a otra, el sistema determina la ruta mas eficiente usando Dijkstra. Cada sucursal involucrada gestiona los productos mediante 3 colas:

- **Cola de ingreso:** Donde llegan los productos recién recibidos. Si la sucursal es el destino final, el producto permanece aquí.
- **Cola de traspaso:** Aplica solo si la sucursal es intermediaria. Los productos que deben continuar hacia otra sucursal se colocan aquí mientras se preparan para el envío.
- **Cola de salida:** Contiene todos los productos listos para ser enviados. Se respeta el intervalo de despacho configurado por sucursal.

El calculo del ETA (Estimated Time of Arrival) considera: el peso de cada arista en la ruta (tiempo o costo), el tiempo de ingreso de cada sucursal intermedia y el tiempo de preparacion de traspaso de cada sucursal intermedia.

Visualizacion

El sistema genera archivos .dot para los 3 arboles (AVL, B y B+) y para el grafo de sucursales, que son renderizados directamente en el navegador usando Viz.js (Graphviz compilado a WebAssembly). Cada usuario puede seleccionar una sucursal y visualizar sus estructuras, descargando los archivos .dot y las imagenes .png correspondientes.

Los archivos generados incluyen el ID de la sucursal en el nombre para evitar sobreescrituras entre sucursales:

- AVL: nombre del producto y altura del nodo en cada nodo del arbol.
- Arbol B: fechas de caducidad ordenadas dentro de cada nodo del arbol.
- Arbol B+: categorias en nodos internos (navegacion) y categoria + nombre en hojas (datos). Las hojas tienen color diferente.
- Grafo: sucursales como nodos con su nombre e ID, aristas etiquetadas con tiempo y costo.

La tabla hash se visualiza con barras en canvas HTML5: verde para buckets con 1 elemento, rojo para colisiones y gris para buckets vacios. Se muestran estadisticas de factor de carga, buckets usados y cadena mas larga.

TADs

TAD Producto

Representa un producto del supermercado. Es la unidad basica de informacion que se almacena en todas las estructuras de datos. El codigo de barras es el identificador unico de cada producto.

Atributos

- **String nombre:** Nombre del producto. Se usa como clave en el Arbol AVL y la Lista Ordenada.
- **String codigoBarras:** Codigo unico del producto. Se usa como clave en la Tabla Hash y para eliminaciones.
- **String categoria:** Categoria del producto. Se utiliza como clave en el Arbol B+.
- **String fechaExpiracion:** Fecha de caducidad en formato YYYY-MM-DD. Clave para el Arbol B y busquedas por rango.
- **String marca:** Marca del producto.
- **Double precio:** Precio del producto en quetzales.
- **Int stock:** Cantidad disponible en el inventario.
- **String estado:** Estado actual: Disponible, En transito o Agotado.
- **String sucursalId:** ID de la sucursal donde esta registrado el producto.

Metodos publicos

- **constructor(nombre, codigo, cat, fecha, marca, precio, stock):** Constructor con parametros. Inicializa todos los campos del producto.
- **equals(otro):** Compara dos productos por nombre. Retorna true si los nombres son iguales.
- **lessThan(otro):** Compara por nombre en orden alfabetico. Retorna true si este producto es menor.
- **greaterThan(otro):** Compara por nombre en orden alfabetico inverso.
- **estaAgotado():** Retorna true si el stock es cero o menor.
- **estaExpirado():** Retorna true si la fecha de caducidad ya paso respecto a la fecha actual.
- **toJSON():** Retorna un objeto plano con todos los campos para persistencia en sessionStorage.
- **fromCSV(linea) [estatico]:** Crea un Producto parseando una linea del CSV de productos.
- **fromForm() [estatico]:** Crea un Producto leyendo los valores del formulario HTML.

TAD Nodo

Nodo generico para las listas enlazadas simples (ordenada y no ordenada). Contiene un Producto y un puntero al siguiente nodo en la cadena.

Atributos

- **Producto data:** El producto almacenado en este nodo.
- **Nodo siguiente:** Referencia al siguiente nodo de la lista. Es null si es el ultimo nodo.

Metodos publicos

- **constructor(producto):** Almacena el producto en data e inicializa siguiente como null.

TAD ListaProductos

Lista enlazada simple no ordenada. Los productos se insertan al frente en $O(1)$. Sirve como estructura base comparativa para medir el costo de la busqueda secuencial frente a estructuras mas eficientes.

Atributos

- **Nodo cabeza:** Referencia al primer nodo de la lista. Es null si la lista esta vacia.
- **Int size:** Contador del numero total de productos en la lista.

Metodos publicos

- **insertar(producto):** Crea un nuevo nodo con el producto y lo coloca al frente de la lista. Complejidad $O(1)$.
- **obtenerPorNombre(nombre):** Recorre la lista comparando nombres. Retorna el producto o null. Complejidad $O(n)$.
- **obtenerPorBarra(codigo):** Recorre la lista comparando codigos de barra. Retorna el producto o null. Complejidad $O(n)$.
- **remove(codigo):** Busca el nodo con ese codigo, lo desenlaza y libera. Complejidad $O(n)$.
- **toArray():** Recorre toda la lista y retorna los productos como array para el frontend.
- **getSize():** Retorna el valor de size.
- **estaVacia():** Retorna true si cabeza es null.
- **limpiar():** Vacía la lista completamente.

TAD ListaProductosOrdenada

Variante de la lista enlazada que mantiene los productos ordenados alfabeticamente por nombre. Permite corte temprano en búsquedas.

Atributos

- **Nodo cabeza:** Referencia al primer nodo de la lista (menor alfabeticamente).
- **Int size:** Contador del numero total de productos.

Metodos publicos

- **insertar(producto):** Recorre la lista para encontrar la posicion correcta e inserta manteniendo el orden. $O(n)$.
- **buscarPorNombre(nombre):** Búsqueda con salida anticipada: si el nodo actual supera alfabeticamente al buscado, retorna null. $O(n)$.
- **buscarPorCodigo(codigo):** Búsqueda secuencial por código de barras. $O(n)$.
- **remove(codigo):** Elimina el nodo con ese código manteniendo el enlace de la lista. $O(n)$.
- **toArray():** Retorna los productos en orden alfabético como array.

TAD AVLNode

Nodo especializado para el árbol AVL. Almacena el Producto, referencias a hijos izquierdo y derecho, y la altura del subárbol para calcular el factor de balance.

Atributos

- **Producto data:** El producto almacenado en este nodo del árbol.
- **AVLNode izquierda:** Referencia al hijo izquierdo (productos con nombre menor).
- **AVLNode derecha:** Referencia al hijo derecho (productos con nombre mayor).
- **Int altura:** Altura del subárbol enraizado en este nodo. Una hoja tiene altura 1.

Metodos publicos

- **constructor(producto):** Almacena el producto, inicializa izquierda y derecha como null, y altura como 1.

TAD ArbolAVL

Arbol binario de busqueda auto-balanceado con clave por nombre del producto. Garantiza que el factor de balance de cada nodo sea -1, 0 o +1 mediante rotaciones automaticas.

Atributos

- **AVLNodo raiz**: Referencia a la raiz del arbol. Es null si el arbol esta vacio.
- **Int size**: Cantidad total de productos almacenados en el arbol.

Metodos publicos

- **insertar(producto)**: Inserta en posicion correcta segun nombre (BST), actualiza alturas y aplica rotaciones si se desbalancea. Rechaza nombres duplicados. $O(\log n)$.
- **buscar(nombre)**: Busqueda binaria: en cada nodo decide si ir a izquierda o derecha. $O(\log n)$.
- **remove(nombre)**: Elimina usando sucesor in-orden para nodos con 2 hijos, luego rebalancea. $O(\log n)$.
- **inOrden()**: Recorrido in-order que retorna los productos en orden alfabetico como array. $O(n)$.
- **generarDot()**: Genera y retorna un string DOT para visualizacion con Viz.js.
- **getSize()**: Retorna el numero de productos en el arbol.
- **estaVacio()**: Retorna true si la raiz es null.

Metodos privados

- **_insertarPriv(nodo, producto, resultado)**: Insercion recursiva con actualizacion de alturas y rotaciones.
- **_removePriv(nodo, nombre, resultado)**: Eliminacion recursiva con rebalanceo en el camino de vuelta.
- **_rotacionIzquierda(x)**: Rotacion simple izquierda para caso RR.
- **_rotacionDerecha(y)**: Rotacion simple derecha para caso LL.
- **_getBalance(nodo)**: Retorna la diferencia de alturas izquierda - derecha.
- **_getAltura(nodo)**: Retorna la altura del nodo o 0 si es null.

TAD BNode

Nodo del arbol B. Puede contener multiples claves (productos) y multiples hijos. La cantidad de claves varia entre $t-1$ y $2t-1$ donde t es el grado minimo.

Atributos

- **Array claves:** Arreglo de productos ordenados por fecha de caducidad. Tamano maximo: $2t-1$.
- **Array hijos:** Arreglo de referencias a nodos hijos. Tamano maximo: $2t$. Null en hojas.
- **Int numClaves:** Cantidad actual de claves almacenadas en este nodo.
- **Int grado:** Grado minimo del arbol ($t=3$). Define los limites de claves e hijos.
- **Boolean esHoja:** Indica si el nodo es una hoja (no tiene hijos).

Metodos publicos

- **constructor(grado, esHoja):** Inicializa los arreglos de claves e hijos con null, numClaves en 0.

TAD ArbolB

Arbol B de grado minimo $t=3$ con clave por fecha de caducidad. Todas las hojas estan al mismo nivel. Cuando un nodo se llena ($2t-1$ claves), se divide y la clave del medio sube al padre.

Atributos

- **BNode raiz:** Referencia a la raiz del arbol.
- **Int grado:** Grado minimo del arbol ($t=3$).
- **Int size:** Cantidad total de productos insertados.

Metodos publicos

- **insertar(producto):** Inserta con division de nodos cuando estan llenos. $O(\log n)$.
- **buscar(fecha):** Busca un producto por fecha exacta. $O(\log n)$.
- **rangoBusqueda(fechaInicio, fechaFin):** Retorna todos los productos en el rango de fechas como array. $O(\log n + k)$.
- **remove(fecha):** Elimina con las 3 estrategias: predecesor, sucesor y fusion. $O(\log n)$.
- **generarDot():** Genera y retorna string DOT para visualizacion.
- **getSize():** Retorna el numero de productos.
- **estaVacio():** Retorna true si la raiz es null.

TAD BMasNodo

Nodo del arbol B+. Los nodos internos solo tienen claves de navegacion (strings de categoria), mientras que las hojas almacenan los datos completos (Productos) y estan enlazadas entre si.

Atributos

- **Array claves:** Arreglo de categorias como claves de navegacion. Tamano maximo: $2t-1$.
- **Array hijo:** Arreglo de referencias a nodos hijos. Solo en nodos internos (null en hojas).
- **Array data:** Arreglo de productos completos. Solo en hojas (null en nodos internos).
- **BMasNodo siguiente:** Referencia al siguiente nodo hoja. Forma la lista enlazada de hojas.
- **Int numClaves:** Cantidad actual de claves en este nodo.
- **Int grado:** Grado minimo del arbol ($t=3$).
- **Boolean esHoja:** Indica si es una hoja (contiene datos) o nodo interno (solo navegacion).

Metodos publicos

- **constructor(grado, esHoja):** Si es hoja crea arreglos de claves y data; si es interno, crea arreglos de claves e hijos.

TAD ArbolBMas

Arbol B+ de grado minimo $t=3$ con clave por categoria. Los datos residen exclusivamente en las hojas enlazadas. Permite claves duplicadas para multiples productos de la misma categoria.

Atributos

- **BMasNodo raiz:** Referencia a la raiz del arbol.
- **Int grado:** Grado minimo del arbol ($t=3$).
- **Int size:** Cantidad total de productos insertados.

Metodos publicos

- **insertar(producto):** Inserta siempre en hoja, sube clave al padre si esta llena. $O(\log n)$.
- **buscarPorCategoria(categoria):** Navega hasta la hoja correcta y recorre el enlace horizontal. Retorna array. $O(\log n + k)$.
- **listarCategorias():** Recorre todas las hojas enlazadas y retorna todos los productos. $O(n)$.

- **remove(categoria, codigo):** Elimina de hoja con fusion o redistribucion si es necesario. $O(\log n)$.
- **generarDot():** Genera y retorna string DOT. Hojas y nodos internos con colores diferentes.

TAD NodoHash

Nodo para la tabla hash con encadenamiento. Cada bucket de la tabla contiene una lista enlazada de NodoHash para manejar colisiones.

Atributos

- **Producto producto:** El producto almacenado en este nodo del bucket.
- **NodoHash siguiente:** Referencia al siguiente nodo en la cadena del mismo bucket.

Metodos publicos

- **constructor(producto):** Almacena el producto e inicializa siguiente como null.

TAD TablaHash

Tabla hash con encadenamiento para manejo de colisiones. La clave es el codigo de barras. Capacidad de 1009 (numero primo) para reducir colisiones.

Atributos

- **Array tabla:** Arreglo de referencias a cadenas de NodoHash. Cada posicion es un bucket.
- **Int capacidad:** Tamano del arreglo (1009 por defecto, numero primo).
- **Int size:** Cantidad total de productos insertados en la tabla.

Metodos publicos

- **insertar(producto):** Calcula hash del codigo e inserta al inicio de la cadena. Verifica duplicados primero. $O(1)$ promedio.
- **buscar(codigo):** Calcula hash y recorre la cadena del bucket buscando el codigo. $O(1)$ promedio.
- **remove(codigo):** Calcula hash, encuentra el nodo en la cadena y lo desenlaza. $O(1)$ promedio.
- **getEstadisticas():** Calcula y retorna objeto con factor de carga, buckets usados, cadena maxima y promedio.
- **toArray():** Recorre todos los buckets y retorna todos los productos como array.
- **getLoadFactor():** Retorna $\text{size} / \text{capacidad}$.

Metodos privados

- **_funcionHash(llave):** Hash polinomial: $\text{hash} = (\text{hash} * 31 + \text{charCode}(c)) \% \text{capacidad}$ para cada caracter.

TAD QueueNode

Nodo para la cola FIFO. Contiene el dato almacenado y una referencia al siguiente nodo.

Atributos

- **Object data:** El dato almacenado (generalmente un Producto).
- **QueueNode siguiente:** Referencia al siguiente nodo. Null si es el ultimo.

Metodos publicos

- **constructor(data):** Almacena el dato e inicializa siguiente como null.

TAD Queue (Cola)

Cola FIFO implementada con lista enlazada. Usada para las 3 colas de despacho de cada sucursal: ingreso, traspaso y salida.

Atributos

- **QueueNode frente:** Referencia al primer nodo de la cola.
- **QueueNode fin:** Referencia al ultimo nodo de la cola.
- **Int size:** Cantidad de elementos en la cola.

Metodos publicos

- **enqueue(data):** Agrega un elemento al final de la cola. $O(1)$.
- **dequeue():** Extrae y retorna el elemento del frente. $O(1)$.
- **peek():** Consulta el frente sin extraerlo. $O(1)$.
- **estaVacia():** Retorna true si la cola esta vacia.
- **getSize():** Retorna el numero de elementos.
- **toArray():** Retorna todos los elementos como array para visualizacion.
- **limpiar():** Vacía la cola completamente.

TAD StackNode

Nodo para la pila LIFO. Contiene el dato y una referencia al nodo anterior en la pila.

Atributos

- **Object data:** El dato almacenado (operacion de rollback).
- **StackNode anterior:** Referencia al nodo anterior en la pila.

Metodos publicos

- **constructor(data):** Almacena el dato e inicializa anterior como null.

TAD Stack (Pila)

Pila LIFO implementada con lista enlazada. Usada por el Catalogo para gestionar el historial de operaciones de rollback.

Atributos

- **StackNode tope:** Referencia al nodo en el tope de la pila.
- **Int size:** Cantidad de elementos en la pila.

Metodos publicos

- **push(data):** Agrega un elemento al tope de la pila. $O(1)$.
- **pop():** Extrae y retorna el elemento del tope. $O(1)$.
- **peek():** Consulta el tope sin extraerlo. $O(1)$.
- **estaVacia():** Retorna true si la pila esta vacia.
- **getSize():** Retorna el numero de elementos.
- **toArray():** Retorna los elementos del tope hacia abajo como array.

TAD Arista

Representa una conexion entre dos sucursales en el grafo ponderado. Almacena el destino y los pesos de la conexion.

Atributos

- **String destino:** ID de la sucursal destino de esta arista.
- **Double tiempo:** Peso en segundos del traslado entre las dos sucursales.
- **Double costo:** Peso en quetzales del traslado entre las dos sucursales.
- **Boolean bidireccional:** Indica si la conexion es bidireccional o unidireccional.

Metodos publicos

- **constructor(destino, tiempo, costo, bidireccional):** Inicializa todos los campos de la arista.

TAD Catalogo

Clase central que contiene las 6 estructuras de datos y garantiza que todas almacenen exactamente los mismos productos en todo momento. Implementa insercion atomica con rollback: si una insercion falla en la estructura N, se deshacen las inserciones exitosas en las estructuras 1 a N-1.

Atributos

- **ListaProductos listaProductos:** Lista enlazada no ordenada con todos los productos.
- **ListaProductosOrdenada listaOrdenada:** Lista enlazada ordenada alfabeticamente por nombre.
- **ArbolAVL arbolAVL:** Arbol AVL indexado por nombre del producto.
- **TablaHash tablaHash:** Tabla hash indexada por codigo de barras.
- **ArbolB arbolB:** Arbol B indexado por fecha de caducidad.
- **ArbolBMas arbolBMas:** Arbol B+ indexado por categoria.
- **Stack pilaDeshacer:** Pila con el historial de operaciones para rollback.
- **Int totalProductos:** Contador del total de productos registrados.

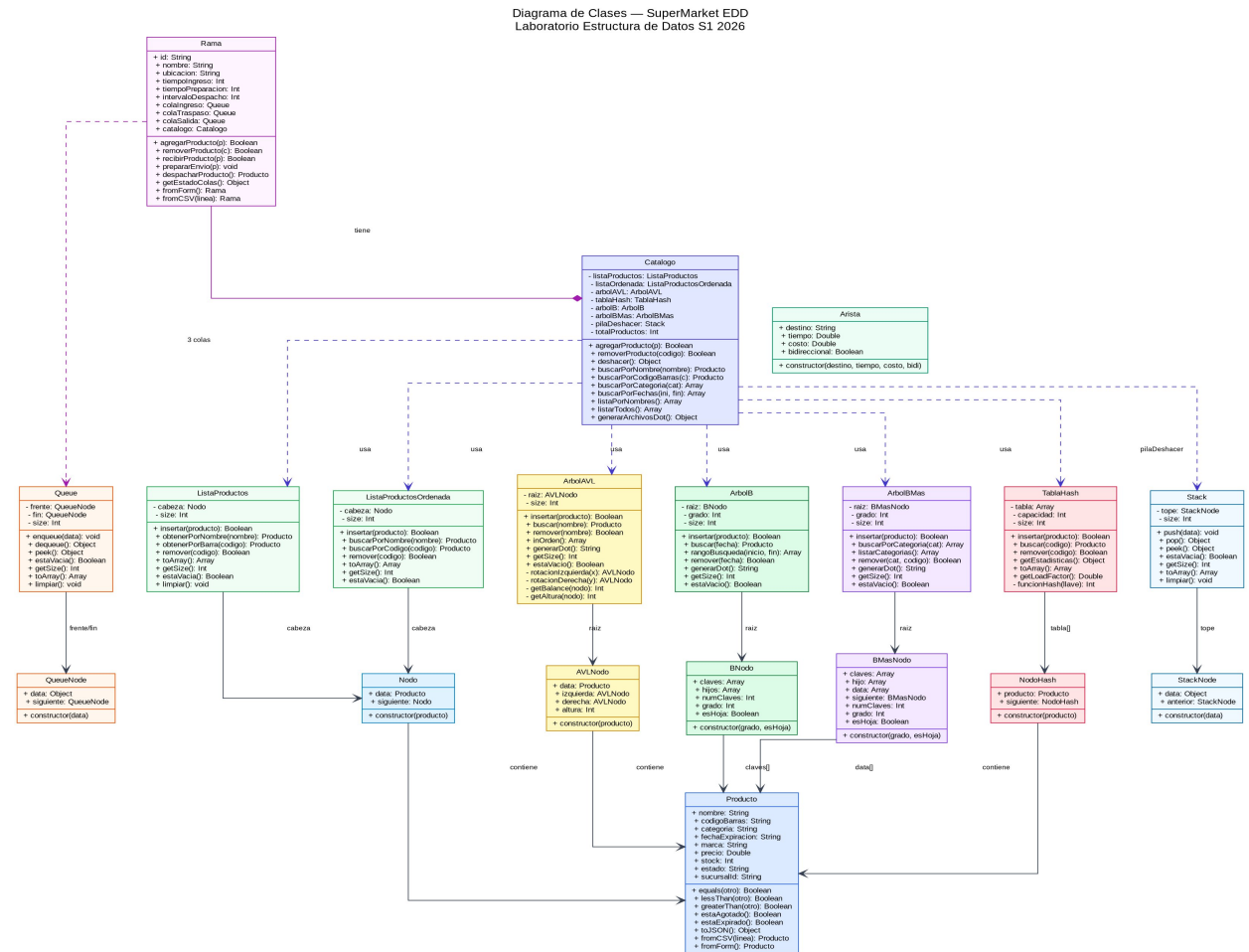
Metodos publicos

- **agregarProducto(producto):** Insercion atomizada en las 6 estructuras con rollback coherente si alguna falla. $O(\log n)$.
- **removeProducto(codigo):** Elimina el producto de las 6 estructuras simultaneamente. $O(\log n)$.

- **deshacer()**: Extrae la ultima operacion de la pila y la revierte en todas las estructuras.
- **buscarPorNombre(nombre)**: Delega al ArbolAVL. $O(\log n)$.
- **buscarPorCodigoBarras(codigo)**: Delega a la TablaHash. $O(1)$ promedio.
- **buscarPorCategoria(categoria)**: Delega al ArbolBMas. $O(\log n + k)$.
- **buscarPorFechas(inicio, fin)**: Delega al ArbolB. $O(\log n + k)$.
- **listaPorNombres()**: Delega al ArbolAVL inOrden. Retorna array ordenado.
- **listarTodos()**: Retorna todos los productos desde la lista no ordenada.
- **generarArchivosDot()**: Retorna objeto con los strings DOT de los 3 arboles.

Diagrama de Clases

El siguiente diagrama muestra todas las clases implementadas y sus relaciones:



Las relaciones del diagrama son:

- Línea sólida con diamante — composición (Rama contiene Catalogo, Catalogo contiene las estructuras)
- Línea sólida con flecha abierta — asociación (ArbolAVL tiene AVLNode como raíz)
- Línea punteada — dependencia de uso (Catalogo usa las 6 estructuras)